Adarsh Saurabh

+91 7004428313 adarshsaurabh1396@gmail.com LinkedIn GitHub Devpost LeetCode

Education

National Institute Of Technology, Rourkela **2025 – 2027**
Master of Technology in Signal and Image Processing, CGPA: 8.28 *Rourkela, India*

Guru Ghasidas University, Bilaspur **2020 – 2024**
Bachelor of Technology in Computer Science and Engineering, CGPA: 8.5 *Bilaspur, India*

Jawahar Navodaya Vidyalaya, Sitamarhi **2017**
Matriculation CBSE, CGPA: 8.8 *Sitamarhi, India*

Relevant Coursework

- Data Structures
- Algorithms Analysis
- Operating Systems
- Database Management
- Software Methodology
- Computer Architecture
- Artificial Intelligence
- Computer Vision

Experience

Autobot Robotics **May 2023 – Aug 2023**
Machine Learning Intern *Remote*

- Mentored and guided a junior intern, imparting mastery in ML fundamentals, Data Manipulation, Numpy, and Pandas.
- Developed and deployed (Django) 4 ML/DL projects with models that may be integrated into company robots.

Ziroh Labs Source Link **Jan 2022 – Feb 2022**
Team Leader - Academic Alliance Program *Remote*

- Fostered strong mentor-team connections, facilitating seamless communication and collaboration.
- Successfully led a team of 5 to achieve project completion ahead of schedule, consistently meeting deadlines.

Key Projects

Uplan — Adversarial Document Intelligence Pipeline Source Link | *Python, Gemini API, LangGraph, Streamlit* **2025**

- Architected an adversarial multi-agent workflow in LangGraph using modern AI tools (Gemini 2.5 Pro specialists) to check visa document coherence; reduced manual verification workload by 85% with detailed failure rebuttal generation.
- Designed structural encoding system using Gemini 2.0 Flash to compile page-level document metadata into a typed semantic graph (98% token compression) and integrated a zero-hallucination mathematical rule check engine.

Alternative Data Radar Source Link | *Python, Next.js, SQL, REST APIs, Bright Data, LLM APIs* **2026**

- Engineered an automated data collection backend in Next.js to gather public web signals (careers and pricing data), routing requests via Bright Data Web Unlocker to bypass anti-scraping mechanisms.
- Implemented AI-driven analysis to calculate a 0–100 corporate health score stored in a SQL database, presenting pre-earnings intelligence and warning alerts in a responsive Recharts dashboard.

IBYD Technology — Warehouse PathMapper Source Link | *Python, Algorithms, Data Structures* **2023**

- Delivered a dynamic warehouse mapping solution as a freelance engagement, working directly with the client to translate their operational pathfinding needs into a heuristic-based routing system.
- Scaled the routing engine to warehouses as large as 10,000×10,000 units, computing optimal paths through 10,000+ locations simultaneously in under 0.5 seconds on a standard i5 laptop CPU.

Achievements

Intermediate Winner — Awaiken Hacks 2023 Source Link: Innovative tool using Mediapipe for real-time action analysis in education with precise feedback on posture and motion

Rank 1 — Girlscript Bilaspur Hackathon "Hack4CG" Source Link: Won top honors for Facial Emotion Recognition project addressing depression vulnerability in modern society

Leadership / Extracurricular

Google Developers Student Club **3rd Year**
Vice President *Guru Ghasidas University*

- Organized technical workshops and hackathons on Machine Learning, Cloud, and Web technologies for 500+ students.

Technical Skills

Languages: Python, C++, C, SQL, JavaScript, HTML, CSS
Core CS & Software Engineering: Data Structures & Algorithms, Object-Oriented Programming, Database Management, REST APIs, Software Methodology, Git, GitHub
Machine Learning & AI: Machine Learning, Deep Learning, Natural Language Processing, Large Language Models, Computer Vision, Multi-Agent Systems, LangGraph, Gemini API, TensorFlow, Scikit-learn, OpenCV, NumPy, Pandas
Tools & Frameworks: VS Code, Anaconda, Django, Flask, Next.js, React Vite, Streamlit

1. Executive Intel, Role, CTC & Live Coding Protocol

Company Profile & Engineering Scale

Media.net is a global pioneer in contextual advertising and sell-side programmatic technology (Directi legacy, founded by Divyank Turakhia; $900M exit). Media.net exclusively powers the **Yahoo! Bing Network** contextual ads program and manages high-quality ad supply on **over 500,000 publisher websites** worldwide (e.g., Forbes, CNN, Reuters).

- **Engineering Constraint**: Real-Time Bidding (RTB) auctions execute under a **hard sub-50ms SLA**.
- **Hiring Philosophy**: High autonomy, extreme meritocracy, and rigorous **unassisted problem-solving**.

Role & Compensation Structure

- **Role**: Software Development Engineer (SDE) Intern (6 Months + Pre-Placement Offer).
- **Monthly Stipend**: **₹1,00,000 / month** (1 LPM).
- **CTC on PPO Conversion**: **₹38.00 LPA Total Compensation**:
  - **Fixed Base Salary**: ₹18.00 Lakhs
  - **One-Time Joining Bonus**: ₹4.00 Lakhs
  - **Additional Bonus**: ₹16.00 Lakhs

The 5-Step Live Coding Protocol (Google Docs / Codeshare)

Media.net technical rounds are conducted without compilers or IDE autocomplete. Follow this protocol:

1. **Clarify Constraints Immediately**: Ask about input boundaries ($N \leq 10^5$), negative values, memory limits, and empty inputs.
2. **State the Brute-Force Baseline First**: Explicitly state the $O(N^2)$ or $O(2^N)$ approach to prove baseline correctness before optimizing.
3. **Derive the Optimal Algorithm**: Explain the optimization ($O(N \log N)$ or $O(N)$) and wait for the interviewer's acknowledgment before typing code.
4. **Dry-Run with Edge Cases**: Manually trace an example and test corner cases (single element, empty array, duplicates, large numbers).
5. **State Complexities Explicitly**: Formulate exact Big-O Time and Auxiliary Space complexities.

Three Fatal Interview Traps

- **Integer Overflow**: Always use `long long` in C++ for running sums or products. Apply modulo ($10^9 + 7$) at every step.
- **Jumping to Code Too Fast**: Candidates who type immediately without discussing trade-offs fail the communication metric.
- **Vague Project Explanations**: Never claim an AI tool or library did the work. Emphasize deterministic logic and ownership.

2. Operating Systems: Deep Systems Mechanics

Process vs. Thread: Low-Level Reality

- **Process**: An executing instance of a program with its own private **Virtual Address Space** (Code, Data/BSS, Heap, Stack), page table hierarchy, file descriptors, and Process Control Block (PCB).
  - *Context Switch Cost*: High ($\approx$ 1–3 μs). Invalidates the CPU Translation Lookaside Buffer (TLB), triggering pipeline stalls and cache misses across L1/L2.
- **Thread**: The smallest schedulable execution unit inside a process. All threads share the text segment, data segment, heap, and open file descriptors, but each thread has its own private **Stack**, **Program Counter (PC)**, and registers (stored in a Thread Control Block).
  - *Context Switch Cost*: Low ($\approx$ 100–300 ns). Page tables remain mapped; TLB entries remain valid.

Virtual Memory, Multi-Level Paging & TLB

- **Virtual Address Translation**: On x86-64, the Memory Management Unit (MMU) uses the CR3 register to traverse 4-level page tables (PML4 $\rightarrow$ PDP $\rightarrow$ PD $\rightarrow$ PT $\rightarrow$ Physical Frame).
- **Translation Lookaside Buffer (TLB)**: An on-chip associative cache of recent virtual-to-physical translations. A TLB hit resolves in < 1 ns; a TLB miss costs $\approx$ 10–50 ns for hardware page table walks.
- **Page Fault**: Occurs via hardware interrupt when accessing a virtual address whose page table entry has the Present bit $= 0$. The kernel traps, reads the 4KB page from disk swap space into RAM, updates the page table, and restarts the instruction.

Concurrency: Mutex vs. Spinlock vs. Counting Semaphore

Mechanism	Blocking / Waiting Behavior	When to Use in Ad-Tech Systems
Mutex	Puts thread to sleep via OS kernel (`futex` wait); yields CPU.	Critical sections > 500 ns, disk I/O, network socket calls.
Spinlock	Busy-waits in user space burning CPU cycles (`PAUSE` loop).	Ultra-low-latency in-memory RTB locks lasting < 50 ns.
Semaphore	Maintains an atomic counter allowing up to N concurrent accesses.	Throttling resource pools (e.g., max 50 concurrent DSP connections).

Deadlocks: The 4 Coffman Conditions & Prevention

A deadlock can occur **if and only if** all four conditions hold simultaneously:

1. **Mutual Exclusion**: Non-shareable resource ownership.
2. **Hold and Wait**: A process holds at least one resource while waiting to acquire others.
3. **No Preemption**: Resources cannot be forcibly revoked until the holder releases them.
4. **Circular Wait**: A closed chain exists where P_0 waits for P_1, ..., P_n waits for P_0.

Production Prevention Strategy: Enforce a strict **global lock acquisition hierarchy** (always acquire Lock A before Lock B) to break the Circular Wait condition.

3. Computer Networks & Database Systems Internals

Computer Networks: Low-Latency Protocols & Socket SRE

- **Why TCP over UDP in Ad-Tech?**: Even though UDP has zero handshake overhead, financial transactions (ad impressions where advertisers are billed millions of dollars) require guaranteed packet delivery and ordering. Losing bid responses causes revenue loss and billing discrepancies.
- **Mitigating Handshake Latency**: Media.net utilizes **Persistent Keep-Alive Connection Pools** and HTTP/2 multiplexing. The 1-RTT TCP handshake is executed once at server boot, eliminating connection setup penalties during live auctions.
- **The `TIME_WAIT` Danger at Scale**: During connection teardown, the active closing endpoint enters `TIME_WAIT` for $2 \times \text{MSL}$ ($\approx 60\,\text{s}$) to absorb duplicate packets. In an exchange handling 50,000 req/sec, closing connections naively exhausts all ephemeral ports (1024–65535), throwing `EADDRNOTAVAIL`. Fixed via `SO_REUSEADDR`, connection pooling, and tuning `net.ipv4.tcp_tw_reuse=1`.
- **HTTP/1.1 vs. HTTP/2**: HTTP/1.1 suffers from Head-of-Line (HoL) blocking at the application layer. HTTP/2 introduces binary framing and stream multiplexing over a single TCP socket.

DBMS Internals: B+ Tree Indexing & Storage Engines

- **Why B+ Trees Beat B-Trees for Disk & SSD**:
  1. *Higher Fan-out*: Internal nodes store only search keys and child pointers (no data payloads). Thousands of keys fit in a single 16KB disk page, keeping tree height shallow ($h \leq 3$ for millions of records).
  2. *Sequential Leaf Scans*: Leaf nodes form a doubly-linked list. Range queries (`BETWEEN t1 AND t2`) require one $O(\log N)$ traversal to find the start key, followed by linear linked-list scanning, avoiding random disk I/O.
- **Clustered vs. Secondary (Non-Clustered) Index**:
  - *Clustered Index*: Leaf nodes *are* the actual physical data pages sorted by Primary Key. Only 1 per table.
  - *Secondary Index*: Leaf nodes store the indexed key plus the primary key pointer.
  - *Bookmark Lookup*: Querying via secondary index requires two lookups (secondary index $\rightarrow$ primary key $\rightarrow$ row data). Avoided using a **Covering Index** where all requested columns reside in the secondary index itself.

ACID Transactions & The 4 SQL Isolation Levels

Isolation Level	Dirty Read	Non-Repeatable	Phantom Read	Underlying Mechanism
Read Uncommitted	Yes	Yes	Yes	Read raw uncommitted memory; no shared locks.
Read Committed	No	Yes	Yes	MVCC: fresh snapshot read per statement.
Repeatable Read	No	No	No (in InnoDB)	MVCC: snapshot read at first statement; Next-Key locks.
Serializable	No	No	No	Strict 2-Phase Locking (2PL); shared locks on ranges.

High-Yield SQL Query: N-th Highest Salary / Bid

```sql
WITH RankedBids AS (
    SELECT bidder_id, bid_price,
           DENSE_RANK() OVER (ORDER BY bid_price DESC) as price_rank
    FROM dsp_bids
)
SELECT DISTINCT bid_price FROM RankedBids WHERE price_rank = 2; -- 2nd Highest Price
```

4. Candidate Resume Defense & Grilling Traps

Project 1: Warehouse PathMapper (IBYD Technology)

Interviewer Question: *"You claim optimal pathfinding on a* $10,000 \times 10,000$ *grid in* $< 0.5\,s$*. On* 10^8 *cells, Dijkstra will timeout. How did you achieve this?"*

- **The Master Defense**:

1. *Topological Waypoint Abstraction*: In real warehouses, 95% of space consists of fixed rack walls. Rather than searching 10^8 cells, I precomputed a hierarchical topological waypoint graph connecting aisle intersections and picking bays ($\approx 1,500$ nodes).
2. *Bidirectional A**: Used Manhattan distance heuristic $h(n) = |x_1 - x_2| + |y_1 - y_2|$ running from both start and target simultaneously, terminating when search frontiers met.
3. *Contiguous Cache Locality*: Coordinates were stored in flat 1D array buffers, eliminating pointer chasing and L1/L2 CPU cache misses.
4. *Ad-Tech Analogy*: Directly parallels ad exchange routing: instead of evaluating every DSP linearly, we prune target criteria using topological clustering.

Project 2: Uplan — Adversarial Document Intelligence Pipeline

Interviewer Question: *"The Media.net JD explicitly tests unassisted problem-solving. Did you write the logic, or did LLMs do the thinking?"*

- **The Master Defense**:

1. *Deterministic Rule Engine First*: LLMs are non-deterministic and hallucinate when validating numbers. I engineered a zero-hallucination mathematical verification engine in Python that checks bank balances, tax credits, and salary figures against deterministic Boolean rules.
2. *Structural Encoding*: Used Gemini Flash strictly as a compilation extractor to compress page metadata into a typed semantic JSON graph, cutting prompt tokens by 98%.
3. *Adversarial LangGraph Topology*: Created Proposer and Adversary roles orchestrated over a deterministic Directed Acyclic Graph (DAG).
4. *AI as Accelerator*: I designed the state graphs, type definitions, and rule engines; AI was used for synthetic edge-case test generation.

Project 3: Alternative Data Radar

Interviewer Question: *"How did you handle anti-scraping, rate limits, and SQL concurrency?"*

- **The Master Defense**:

1. *Proxy Rotation*: Routed traffic via Bright Data Web Unlocker, managing TLS fingerprint rotation and exponential backoff with jitter on HTTP 429/503.
2. *Async Worker Architecture*: Used Python `asyncio` worker pools to stream parsed signals asynchronously into the database.
3. *SQL Optimization*: Created composite indexes on `(company_id, metric_date DESC)` to ensure constant-time dashboard reads.

Defending Your Dual Background: M.Tech Signal Processing + B.Tech CSE

Interviewer Question: *"Why apply for Core SDE at Media.net with a Master's in Signal Processing?"*

- **The Winning Narrative**:

1. *B.Tech in CSE*: Provided rock-solid foundations in Data Structures, Algorithms, OS, Networks, and Databases.
2. *M.Tech at NIT Rourkela*: Trained me in high-dimensional linear algebra, statistical signal filtering, and real-time streaming pipelines.
3. *Direct Ad-Tech Application*: Programmatic ad bidding is fundamentally a real-time digital signal stream. Click fraud detection relies on statistical anomaly filtering (Kalman filtering); contextual matching uses high-dimensional vector embeddings; and sub-50ms RTB requires microsecond timing budgeting.

5. Ad-Tech Domain Architecture & Low-Level Design

The Sub-50ms Real-Time Bidding (RTB) Latency Budget

Step	Pipeline Operation	Budget	Underlying Mechanism / Technology
1	Browser → Edge SSP Request	10 ms	HTTP/2, Anycast DNS routing to closest edge PoP
2	Contextual NLP & Keyword Extraction	5 ms	In-memory Trie matcher, cached URL semantic vector
3	OpenRTB Serialization & DSP Dispatch	2 ms	Zero-copy JSON / Protocol Buffers serialization
4	DSP Bid Computation (Parallel Wait)	25 ms	Hard deadline timeout; non-responding DSPs dropped
5	Auction Resolution (Second-Price Vickrey)	3 ms	Min-Heap / QuickSelect top-K selection & pricing
6	Ad Markup Rendering & Async Logging	5 ms	Asynchronous logging to Kafka; return ad markup HTML
Total	**End-to-End Auction Pipeline**	**50 ms**	**Hard Sub-50ms SLA Guarantee**

Header Bidding (`Prebid.js`) vs. Traditional Waterfall

- **Waterfall (Legacy)**: Sequential daisy-chaining where requests cascaded down through SSPs. Caused 500ms–1500ms latency and left money on the table.
- **Header Bidding**: Lightweight JavaScript (`Prebid.js`) in the publisher's page `<head>` makes **concurrent asynchronous calls** to multiple SSPs before calling the ad server, running a transparent, unified auction that maximizes yield by 30–50%.

High-Scale Data Structures in Ad-Tech

- **Frequency Capping**: Restricts how often an ad is shown to a user. Stored efficiently using **Redis Bitmaps** and sliding window deques.
- **HyperLogLog (HLL)**: Estimates unique audience cardinality across billions of impressions using only **1.5 KB of memory** with $\approx 0.81\%$ standard error.
- **Bloom Filters**: In-memory probabilistic filters used at edge proxies to drop known click-fraud bot traffic in $< 10\,\mu s$.

Machine Coding (LLD): Thread-Safe In-Memory Cache with TTL & LRU

```cpp
#include <unordered_map>
#include <list>
#include <mutex>
#include <chrono>
#include <optional>

template <typename K, typename V>
class LRUCacheWithTTL {
    struct Item { K key; V val; std::chrono::steady_clock::time_point exp; };
    size_t cap_;
    std::list<Item> list_; // Front = MRU, Back = LRU
    std::unordered_map<K, typename std::list<Item>::iterator> map_;
    mutable std::mutex mtx_;

public:
    explicit LRUCacheWithTTL(size_t cap) : cap_(cap) {}

    std::optional<V> get(const K& key) {
        std::lock_guard<std::mutex> lock(mtx_);
        auto it = map_.find(key);
        if (it == map_.end()) return std::nullopt;
        if (std::chrono::steady_clock::now() > it->second->exp) {
            list_.erase(it->second); map_.erase(it); return std::nullopt;
        }
        list_.splice(list_.begin(), list_, it->second); // Move to MRU
        return it->second->val;
    }

    void put(const K& key, const V& val, int64_t ttl_ms) {
        std::lock_guard<std::mutex> lock(mtx_);
        auto exp = std::chrono::steady_clock::now() + std::chrono::milliseconds(ttl_ms);
        auto it = map_.find(key);
        if (it != map_.end()) {
            it->second->val = val; it->second->exp = exp;
            list_.splice(list_.begin(), list_, it->second);
            return;
        }
        if (map_.size() >= cap_) {
            map_.erase(list_.back().key); list_.pop_back(); // Evict LRU
        }
        list_.push_front({key, val, exp});
        map_[key] = list_.begin();
    }
};
```

6. Signature Coding Patterns & Rapid Recall CheatSheet

Signature Algorithmic Patterns Tested at Media.net

1. **Top-K Generalized Second-Price (GSP) Auction**: Maintain a Min-Heap of size $K+1$. For N incoming bids, total time is $O(N \log K)$ and space is $O(K)$. The i-th winner pays the $(i+1)$-th bid price.
2. **Contextual Keyword Wildcard Trie**: Prefix tree supporting wildcard `?` (match any single character) and `*` (branch into 0 or more characters).
3. **Weighted Interval Scheduling DP**: Given N ad campaigns with start, end, and revenue, sort by end time. For each interval i, binary search (`std::upper_bound`) for the latest non-overlapping interval $p(i)$. Recurrence: $DP[i] = \max(DP[i-1], V_i + DP[p(i)])$ in $O(N \log N)$.
4. **DAG Pipeline Cycle Detection & Critical Path**: Use Kahn's algorithm (topological sort with in-degrees). If processed nodes $< N$, cycle detected. Critical path latency $= \max_u(\text{completion_time}[u])$.

High-Yield Systems & Queueing Formulas

- **Amdahl's Law (Speedup Limit)**: $S_{\text{latency}} = \frac{1}{(1-p)+\frac{p}{s}}$ (If 10% is serial, speedup cannot exceed 10× even with ∞ cores).
- **Little's Law**: $L = \lambda \times W$ (Average requests in system L = Throughput $\lambda\times$ Latency W. For $50,000$ req/s at 50 ms, the system holds $50000 \times 0.05 = 2,500$ in-flight requests).
- **Bandwidth-Delay Product (BDP)**: BDP = Bandwidth (bits/s) × RTT (s) (Determines socket buffer size).

High-Yield C++ Concurrency Snippets

```
// Thread-Safe RAII Lock
std::mutex mtx;
{ std::lock_guard<std::mutex> lock(mtx); }

// Condition Variable Pattern (Producer-Consumer)
std::condition_variable cv;
std::unique_lock<std::mutex> ulock(mtx);
cv.wait(ulock, []{ return !queue.empty(); }); // sleeps & auto-reacquires lock
```

Key Ad-Tech Business Metrics

- **CPM**: Cost Per Mille = Cost per 1,000 ad impressions.
- **eCPM**: $\frac{\text{Total Earnings}}{\text{Total Impressions}} \times 1000$.
- **CTR**: $\frac{\text{Total Clicks}}{\text{Total Impressions}} \times 100\%$.
- **Vickrey Auction (Second-Price)**: The highest bidder wins but pays the second-highest bid price +$0.01. Guarantees truthful bidding as a dominant strategy.

Behavioral Fit: STAR Response Strategy

- **Production Outage Bug**: Structure around **Warehouse PathMapper**: memory allocation spike on 10^4 grid → profiled with Valgrind → refactored to contiguous array buffers → latency dropped from crash to 0.48 s.
- **Technical Disagreement**: Structure around **Ziroh Labs**: ResNet-50 vs. custom unsupervised HOG → ran an objective 24-hr benchmark spike → HOG was 11× faster with 96.45% accuracy → data resolved the dispute.
- **Why Media.net**: Scale (500k+ publishers, billions of requests), pure engineering meritocracy, and microsecond distributed systems aligning with my dual background in CSE and Signal Processing.